1 GERENCIAMENTO DE PROJETOS ÁGEIS

A PharmaStock é uma empresa do ramo farmacêutico criada para auxiliar o desenvolvimento deste projeto. O objetivo do sistema é sobre a gestão de estoque em farmácias, organizando o controle de produtos, fornecedores, lotes, entradas e vendas.

O projeto foi conduzido por meio do gerenciamento ágil de projetos. Para garantir a organização, a previsibilidade e a entrega em etapas, a equipe utiliza uma abordagem híbrida combinando Scrum que é aplicado no planejamento e no acompanhamento de Sprints e o Kanban na ajuda para visualizar a evolução na gestão do fluxo de trabalho.

O processo de gestão abrange o mapeamento dos envolvidos (stakeholders), o levantamento de requisitos, a organização e priorização do Product Backlog, o planejamento das iterações, os critérios de aceite e a gestão de riscos. Todas as ações tomadas seguem as definições alinhadas na reunião de Kickoff do pim anterior, podendo ser ajustadas ao longo do ciclo de desenvolvimento.

1.1 Metodologia adotada

O projeto utiliza Scrum (ciclos curtos de entrega) e Kanban (controle visual de tarefas). Como o sistema envolve banco de dados SQL Server, backend em C#, web e aplicativo móvel em Flutter, entregar o software aos poucos reduz riscos técnicos e evita surpresas no final.

Alinhada ao Manifesto Ágil, a equipe foca em entregas frequentes e colaboração. Assim, as funções mais críticas do controle de estoque são finalizadas primeiro, permitindo melhorias contínuas ao longo do projeto.

1.2 Papéis da equipe

A equipe adota uma organização colaborativa, com responsabilidades distribuídas entre os integrantes conforme suas áreas de atuação. Embora sejam aplicadas práticas do Scrum, as responsabilidades não seguem rigorosamente a divisão fixa de papéis do framework, o que permite a contribuição de todos no fluxo global do projeto.

As atribuições da equipe no desenvolvimento do PharmaStock dividem-se em:

- **Anderson (Núcleo C# e API):** responsável pelas camadas de Modelos, Repositório e Negócio, além do desenvolvimento da API e da integração dos fluxos do sistema.

- **Geovanni (Banco de Dados):** responsável pela estrutura do banco de dados SQL Server, incluindo tabelas, procedures, triggers e a geração de dados para testes.
- **Vinicius (Aplicação Web):** responsável pela interface web em ASP.NET MVC e pela adaptação responsiva.
- **Roberto (Aplicativo Mobile):** responsável pelo desenvolvimento da aplicação móvel em Flutter e pela sua integração com a API.
- **Geovanna (Gerenciamento de Projetos Ágeis):** responsável pelo levantamento de requisitos, organização e acompanhamento do Product Backlog, planejamento das sprints, gestão de riscos e documentação ágil.

Além das atribuições individuais, a integração dos módulos, a validação das funcionalidades e a redação da documentação final constituem responsabilidades coletivas da equipe.

1.3 Partes interessadas (stakeholders)

Partes interessadas são as pessoas ou grupos afetados pelo sistema ou que influenciam o projeto. No PharmaStock, foram identificadas as seguintes:

- **Proprietário ou gerente da farmácia:** Controle confiável do estoque, redução de perdas e relatórios para decisão de compras.
- **Farmacêutico responsável:** Controle de validade dos lotes e identificação de medicamentos que exigem receita.
- **Atendente (usuário final):** Telas simples e ágeis para consulta de produtos e registro de vendas.
- **Operador de estoque:** Consulta rápida de produtos, lotes e validades pelo celular, direto na prateleira.
- **Fornecedores:** Cadastro correto dos dados da empresa e do histórico de entregas.
- **Equipe do projeto e professores:** Organização do trabalho, entregas no prazo e cumprimento das orientações.

1.4 Requisitos funcionais e não funcionais

Os requisitos funcionais e não funcionais foram revisados conforme o escopo definido para o PIM IV e as decisões alinhadas na reunião de Kickoff. As funcionalidades abrangem os processos essenciais da farmácia, tais como autenticação de usuários, cadastro de produtos e fornecedores, controle de lotes, movimentações de entrada, vendas e consultas de estoque. Além disso, as especificações garantem a correta integração entre o banco de dados SQL Server, a aplicação web e o aplicativo móvel.

Requisitos Funcionais

- **RF01 –** Login de funcionários por CPF ou e-mail e senha.
- **RF02 –** Cadastro e consulta de fornecedores.
- **RF03 –** Cadastro de categorias.
- **RF04 –** Cadastro e consulta de produtos.
- **RF05 –** Gerenciamento de lotes.
- **RF06 –** Localização ou criação de lotes durante entradas.
- **RF07 –** Registro de entradas de produtos.
- **RF08 –** Registro de vendas.
- **RF09 –** Controle de saída pelo método FEFO.
- **RF10 –** Impedir vendas acima da quantidade disponível.
- **RF11 –** Validar funcionários, produtos, quantidades e preços nas operações.
- **RF12 –** Consulta de movimentações.
- **RF13 –** Atualização das quantidades dos lotes por gatilhos.
- **RF14 –** Recalcular os totais de entradas e vendas por gatilhos.
- **RF15 –** Listagem de informações na aplicação Web.
- **RF16 –** Consulta de lotes próximos do vencimento.
- **RF17 –** Consulta de produtos com estoque baixo.
- **RF18 –** Login e navegação no aplicativo Flutter.
- **RF19 –** Disponibilização de dados iniciais para demonstração.
- **RF20 –** Bloqueio da venda de produtos vencidos.
- **RF21 –** Cancelamento ou reversão de entradas e vendas.
- **RF22 –** Descontos e tabela de preços.

Requisitos Não Funcionais

- **RNF01 –** Utilização de arquitetura em camadas.
- **RNF02 –** Utilização do SQL Server, com pelo menos um gatilho e uma procedure armazenada executada pelo repositório.
- **RNF03** – Garantia da integridade dos dados.
- **RNF04 –** Utilização de consultas parametrizadas e armazenamento da conexão no appsettings.json.
- **RNF05 –** Identificação dos funcionários no MVP, deixando JWT para etapa posterior.
- **RNF06 –** Apresentação de mensagens de erro compreensíveis.
- **RNF07 –** Utilização de transações em operações com múltiplos itens.
- **RNF08 –** Padronização do esquema do banco de dados e das procedures.
- **RNF09 –** API REST com JSON, OpenAPI e Scalar.
- **RNF10 –** Interface Web responsiva.
- **RNF11 –** Aplicativo Flutter integrado exclusivamente pela API.
- **RNF12 –** Desempenho adequado para a demonstração acadêmica.

- **RNF13 –** Portabilidade do ambiente de desenvolvimento.
- **RNF14 –** Documentação conforme as normas ABNT.
- **RNF15 –** Abordagem sobre acessibilidade e LIBRAS na documentação.
- **RNF16 –** Abordagem sobre análise de dados e aprendizado de máquina.

1.5 Product Backlog e priorização

O Product Backlog reúne todas as entregas do sistema, organizadas no formato de Histórias de Usuário ("Como [quem], quero [o quê], para [por quê]"). A prioridade varia de 1 a 5, considerando o valor de negócio para a farmácia e a dependência técnica entre as tarefas (como criar a API antes do aplicativo móvel).

O Product Backlog foi estruturado com base no Produto Mínimo Viável (MVP, do inglês Minimum Viable Product) definido na reunião de Kickoff . As funcionalidades essenciais, como autenticação de usuários, cadastros básicos, controle de estoque e registro de vendas receberam alta prioridade. Já as consultas complementares, os relatórios e as rotinas adicionais do aplicativo móvel foram planejados para as etapas seguintes, conforme a capacidade de entrega da equipe. Essa priorização pode ser ajustada ao longo do desenvolvimento segundo as dependências técnicas e o valor gerado para o negócio.

- **IT01 (Prioridade 5 | Sprint 1)**
  - **História:** Como equipe, queremos preparar a base técnica (solution com cinco projetos e banco SQL Server com trigger e procedure), para que todas as funcionalidades sejam construídas sobre ela.
  - **Requisitos:** RNF03, RNF05, RNF07
- **US01 (Prioridade 5 | Sprint 1)**
  - **História:** Como funcionário, quero entrar no sistema com usuário e senha, para acessar apenas o que é permitido para mim.
  - **Requisitos:** RF01
- **US02 (Prioridade 5 | Sprint 1)**
  - **História:** Como gerente, quero cadastrar e consultar produtos e fornecedores, para manter o catálogo da farmácia atualizado.
  - **Requisitos:** RF02 a RF04, RF08, RF09
- **US03 (Prioridade 4 | Sprint 2)**
  - **História:** Como gerente, quero cadastrar e inativar funcionários, para controlar quem usa o sistema.
  - **Requisitos:** RF05 a RF07
- **US04 (Prioridade 5 | Sprint 2)**
  - **História:** Como farmacêutico, quero registrar lotes com quantidade e validade, para saber o que está no estoque e quando vence.
  - **Requisitos:** RF10
- **US05 (Prioridade 5 | Sprint 2)**

    - **História:** Como operador de estoque, quero registrar entradas e saídas, para manter o saldo correto e o histórico.
    - **Requisitos:** RF11
- **US06 (Prioridade 4 | Sprint 3)**
    - **História:** Como atendente, quero registrar uma venda com total e desconto, para atender o cliente com rapidez.
    - **Requisitos:** RF13
- **US07 (Prioridade 4 | Sprint 3)**
    - **História:** Como aplicativo móvel, quero uma Web API que forneça produtos e estoque, para consultar os dados do sistema.
    - **Requisitos:** RF14, RNF06
- **US08 (Prioridade 4 | Sprint 4)**
    - **História:** Como operador de estoque, quero consultar produto, lote, validade e quantidade pelo celular, para conferir a prateleira sem usar o computador.
    - **Requisitos:** RF14
- **US09 (Prioridade 3 | Sprint 4)**
    - **História:** Como gerente, quero ver alertas de validade e relatórios de estoque, para decidir compras e evitar perdas.
    - **Requisitos:** RF12, RF15

1.6 Estimativa e iterações

O desenvolvimento do PharmaStock está estruturado em três sprints, visando fracionar as entregas em etapas incrementais e otimizar o acompanhamento do projeto.

- ***Sprint* 1 – Estrutura Técnica e Cadastros Básicos:**
    - Modelagem e criação da estrutura do banco de dados SQL Server;
    - Implementação da autenticação e controle de acesso (login);
    - Desenvolvimento dos cadastros de fornecedores e produtos;
    - Mapeamento e rotinas de gerenciamento de lotes;
    - Configuração da infraestrutura inicial e rotas da Web API.
- **Sprint 2 – Regras de Negócio e Controle de Estoque:**
    - Implementação do registro das movimentações de entradas e vendas;
    - Automação do saldo dos lotes via gatilhos (triggers) no banco de dados;
    - Aplicação da lógica de baixa de estoque por FEFO (First Expired, First Out);
    - Validação das regras operacionais e impedimento de vendas inconsistentes;
    - Desenvolvimento da consulta ao histórico de movimentações.
- ***Sprint* 3 – Interfaces, Integrações e Finalização:**

    - Construção da interface da aplicação web (ASP.NET MVC);
    - Desenvolvimento do aplicativo móvel em Flutter e integração com a Web API;
    - Implementação dos relatórios e alertas de lotes próximos ao vencimento e estoque crítico;
    - Execução dos testes integrados do sistema e preparação do ambiente para demonstração.

A gestão temporal e o detalhamento das atividades por integrante são acompanhados e atualizados continuamente por meio da ferramenta Kanban no Trello.

1.7 Critérios de aceite e definição de pronto

Os critérios de aceite definem as condições necessárias para considerar uma história concluída. Além dos critérios individuais de cada item, o grupo utiliza uma Definição de Pronto (Definition of Done): uma tarefa só é finalizada quando o código funciona sem erros, passa por testes manuais, cumpre os critérios de aceite, passa por revisão e é publicado no GitHub.

Critérios de Aceite por Item

- **IT01:** A solução compila com os 5 projetos (Modelos, Repositório, Negócio, Web e API), mantendo as dependências no sentido correto (`Web/API → Negócio → Repositório → Modelos`). O banco utiliza ao menos uma *trigger* e uma *procedure* (esta executada via sistema).
- **US01:** Autentica dados válidos, exibe mensagem clara para erros e salva senhas com hash (criptografadas).
- **US02:** Permite cadastrar, listar e remover produtos e fornecedores, destacando os medicamentos que exigem receita médica.
- **US03:** Impede a duplicidade de CPF no cadastro de funcionários e preserva os dados dos inativados no histórico.
- **US04:** Associa quantidade e data de validade a cada lote, exibindo essas informações na listagem.
- **US05:** Atualiza o saldo do produto a cada entrada ou saída e registra a movimentação no histórico.
- **US06:** Calcula o valor total da venda e impede que aplicação de descontos gere saldos negativos.
- **US07:** Disponibiliza endpoints REST que retornam JSON com status HTTP adequados (200, 201, 400, 404 e 500).
- **US08:** Exibe no aplicativo móvel o nome, lote, quantidade e validade do produto consultado via Web API.

- **US09:** Destaca itens com estoque baixo ou validade próxima e permite a visualização de ao menos um relatório.

1.8 Gerenciamento de riscos

Risco é um evento incerto que, se ocorrer, pode impactar negativamente o projeto. Para gerenciar essas ameaças, a equipe mapeou os principais riscos e definiu ações de resposta (como mitigar, evitar ou aceitar).

Tabela – Riscos do projeto e ações de mitigação

Risco	Resposta	Ação prevista
Prazo curto para entregar Web, API e App	Mitigar	Priorizar funções essenciais e acompanhar o Kanban semanalmente.
Erros na migração para SQL Server	Mitigar	Montar a base técnica na Sprint 1 e testar os scripts desde o início.
Falhas ao gravar em tabelas com trigger	Evitar	Seguir o manual do PIM IV para mapeamento e testar a trigger logo após a criação.
Incompatibilidade entre partes do código	Mitigar	Fazer integrações frequentes no GitHub e revisar o código em dupla.
Ausência/sobrecarga de integrantes	Aceitar	Redistribuir tarefas na reunião diária e reajustar o planejamento da sprint, se preciso.
Incompatibilidade entre as procedures, triggers e a aplicação C#	Mitigar	Realizar testes de integração entre o banco de dados e a aplicação, validando os parâmetros e os resultados das operações.
Atraso na integração entre API, Web e aplicativo Flutter	Mitigar	Planejar as dependências entre as equipes e realizar validações

		progressivas dos endpoints e das funcionalidades integradas.

1.9 Acompanhamento do projeto

O progresso do projeto é acompanhado por um quadro Kanban no Trello, estruturado nas colunas Product Backlog, A Fazer, Fazendo e Feito. Cada história de usuário é representada por um cartão que avança conforme a evolução do trabalho. Para evitar gargalos e manter o fluxo contínuo de entregas, a equipe limita o Trabalho em Andamento (WIP - Work in Progress) na coluna "Fazendo", focando em concluir as tarefas abertas antes de iniciar novas.

Todo o código-fonte e os scripts SQL são versionados no GitHub, garantindo rastreabilidade, histórico de alterações e revisão por pares. Dessa forma, o Kanban fornece a visibilidade do fluxo de trabalho, enquanto o repositório reflete a evolução real do sistema.

O acompanhamento do projeto considera as decisões e pendências registradas na reunião de Kickoff deste pim. Entre as principais atividades pendentes estão:

- Conferência e atualização dos scripts de estrutura (schema.sql) e carga inicial de dados (seed.sql) no SQL Server;
- Acompanhamento do desenvolvimento da aplicação móvel em Flutter, focada na exibição de alertas de estoque crítico e lotes próximos do vencimento;
- Refinamento contínuo das histórias de usuário (User Stories), prioridades e critérios de aceite na ferramenta Trello;
- Gestão do ciclo das sprints e monitoramento das entregas conforme o papel desempenhado por cada integrante.

Essas pendências serão monitoradas ao longo do ciclo de desenvolvimento, respeitando as atribuições de cada integrante.